
A8\code\A8.py

```
import matplotlib.pyplot as plt
import numpy as np
from utils import *

# weitere Module importieren
# BEGIN SOLUTION
from matplotlib import cm
import time
# END SOLUTION

def armijo(f, p, x, rho, c1, alpha0=1):
    """
    Armijo Backtracking Liniensuche

    Input:
        f: Funktion
        p: Vektor (Abstiegsrichtung)
        x: Vektor (Iterierte)
        rho: Skalar (Reduktionsfaktor für alpha)
        c1: Skalar (Parameter für die Armijo-Bedingung)
        alpha0: Skalar (Initial getesteter Startwert)

    Output:
        alpha: Skalar (gesuchte Schrittweite)
    """

    # TODO: Aufgabenteil 1.i: Armijo Backtracking Liniensuche implementieren.
    # BEGIN SOLUTION
    alpha = alpha0
    fx, grad_fx, _ = f(x)
    for i in range(100):
        x_alpha_p = (x + alpha * p).ravel()
        fx_alpha_p, _, _ = f(x_alpha_p)

        if fx_alpha_p <= fx + c1 * alpha * np.inner(grad_fx.T, p):
            break
        else:
            alpha = rho * alpha
    else:
        print("Nach 100 Iterationen wurde keine Schrittweite gefunden, welche die Armijo-Bedingung erfüllt")
```

```

return alpha
# END SOLUTION

def gill_murray_wright(xk, xkplus1, valk, valkplus1, gradk, k, eps, tau, kmax):
    """
    Abbruchkriterien nach Gill, Murray und Wright

    Input:
        xk: Vektor (aktuelle Iterierte)
        xkp1: Vektor (nächste Iterierte)
        valk: Skalar (f(xk))
        valkp1: Skalar (f(xkp1))
        gradk: Vektor ( $\nabla f(xk)$ )
        k: Integer (Iteration-Nummer)
        tau: Skalar (Toleranz für Kriterium 1)
        eps: Skalar (Toleranz für Gradient -- Kriterium 2)
        kmax: Integer (maximale Anzahl der Iterationen)

    Output:
        test: Boolescher Wert (Ergebnis der Abbruchkriterien nach Gill, Murray, Wright)
    """

    # TODO: Aufgabenteil 1.ii: Gill-Murray-Wright-Abbruchkriterien ueberpruefen.
    # BEGIN SOLUTION
    test = False #Initialisiere Boolean

    #Pruefe erste Bedingung
    eins_a_1 = valk - valkplus1
    eins_a_2 = tau * (1 + abs(valk))

    eins_b_1 = np.linalg.norm(xkplus1 - xk)
    eins_b_2 = tau*(1/2) * (1 + np.linalg.norm(xk))

    eins_c_1 = np.linalg.norm(gradk)
    eins_c_2 = tau*(1/3) * (1 + abs(valk))

    eins_a_ungl = eins_a_1 < eins_a_2
    eins_b_ungl = eins_b_1 < eins_b_2
    eins_c_ungl = eins_c_1 < eins_c_2

    if eins_a_ungl and eins_b_ungl and eins_c_ungl:
        test = True
        print("Die Verbesserung der Werte, der xk und der Norm des Gradienten sind sehr klein.\n")

    #Pruefe 2. Bedingung
    if np.linalg.norm(gradk) < eps:
        test = True
        print("Die Norm des Gradienten ist kleiner epsilon.\n")

    #Pruefe 3. Bedingung
    if k >= kmax:
        test = True
        print("Die maximale Anzahl an Iterationen ist erreicht.\n")

    return test

```

```
# END SOLUTION
```

```
def gradient_descent_erweitert(f, x0, rho=0.5, c1=1e-3, eps=1e-10, tau=1e-10, kmax=100, xstar=None):  
    """
```

```
    Gradientenabstiegsverfahren
```

```
    Input:
```

```
        f: Funktion, wobei  $f(x) = (val, grad, hess)$ ,  
        x0: Vektor (Startwert),  
        rho: Skalar (Armijo Parameter für die Anpassung der Schrittweite)  
        c1: Skalar (Armijo Parameter für das Testen der Bedingung),  
        tau: Skalar (Toleranz für Kriterium 1),  
        eps: Skalar (Toleranz für Gradient in GMW-Kriterium 2),  
        kmax: Skalar (Maximale Anzahl an Iterationen),  
        xstar: Vector (Minimierer von f), optional
```

```
    Output:
```

```
        log: Dictionary vom Verlauf der Optimierung.
```

```
    """
```

```
    # Dictionary zum Abspeichern der Ergebnisse.
```

```
    log = {  
        'xstar': xstar,                # Tatsächlicher Minimierer  
        'val_star': None if xstar is None else f(xstar)[0], # Tatsächliches Minimum  
        'x0': x0,                      # Startwert  
        'xgd': None,                   # Lösung des Gradientenabstiegsverfahrens  
        'kgd': None,                   # Anzahl benötigter Iterationen  
        'x_list': [],                  # Liste der Iterierten  
        'val_list': [],                 # Liste des Funktionswerts in den Iterierten  
        'norm_grad_list': [],           # Liste der Norm des Gradienten in den Iterierten  
    }
```

```
    # TODO: Aufgabenteil 1.iii: Funktion ergänzen.
```

```
    # BEGIN SOLUTION
```

```
    # Nutzen Lösungsvorschlag der A5.py zur Erweiterung
```

```
    k = 0
```

```
    xk = x0
```

```
    #verändert
```

```
    valk, gradk, hessk = f(x0)
```

```
    log['x_list'].append(x0)
```

```
    log['val_list'].append(valk)
```

```
    log['norm_grad_list'].append(np.linalg.norm(gradk))
```

```
    while True:
```

```
        # Schritt
```

```
        pk = -gradk
```

```
        #Neu
```

```
        alpha_optimal = np.dot(gradk, gradk) / np.dot(gradk, (hessk @ gradk).ravel())
```

```
        schritt_init = max(1, alpha_optimal)
```

```
        alphak = armijo(f, pk, xk, rho, c1, schritt_init)
```

```
        #alt
```

```
        xkplus1 = xk + alphak * pk
```

```
        #neu
```

```
        valkplus1, gradkplus1, hesskplus1 = f(xkplus1)
```

```

        #Abbruchbedingung
        if gill_murray_wright(xk, xkplus1, valk, valkplus1, gradk, k, eps, tau, kmax):
            break

        #Update
        #alt
        k += 1
        xk = xkplus1
        #neu
        valk = valkplus1
        gradk = gradkplus1
        hessk = hesskplus1

        # Logging - unsortiert
        log['x_list'].append(xk)
        log['val_list'].append(valk)
        log['norm_grad_list'].append(np.linalg.norm(gradk))

log['xgd'] = xk
log['kgd'] = k
# END SOLUTION
return log

def newton_erweitert(f, x0, rho=0.5, c1=1e-3, eps=1e-10, tau=1e-10, kmax=100, xstar=None):
    """
    Newton-Verfahren zur Minimierung der Funktion f.

    Input:
        f: Funktion, wobei  $f(x) = (val, grad, hess)$ ,
        x0: Vektor (Startwert),
        rho: Skalar (Armijo Parameter für die Anpassung der Schrittweite)
        c1: Skalar (Armijo Parameter für das Testen der Bedingung),
        tau: Skalar (Toleranz für Kriterium 1),
        eps: Skalar (Toleranz für Gradient in GMW-Kriterium 2),
        kmax: Skalar (Maximale Anzahl an Iterationen),
        xstar: Vector (Minimierer von f), optional

    Output:
        log: Dictionary vom Verlauf der Optimierung.
    """

    # Dictionary zum Abspeichern der Ergebnisse.
    log = {
        'xstar': xstar,          # Tatsächlicher Minimierer
        'val_star': None if xstar is None else f(xstar)[0], # Tatsächliches Minimum
        'x0': x0,               # Startwert
        'xnv': None,            # Lösung des Newton-Verfahrens
        'knv': None,            # Anzahl benötigter Iterationen
        'x_list': [],           # Liste der Iterierten
        'val_list': [],          # Liste des Funktionswerts in den Iterierten
        'norm_grad_list': [],    # Liste der Norm des Gradienten in den Iterierten
    }

    # TODO: Aufgabenteil 1.iii: Funktion ergänzen.

```

```

# BEGIN SOLUTION
# Nutzen Lösungsvorschlag der A7_2.py zur Erweiterung
xk = x0
k = 0
#angepasst
alphak = 1
#Initiale Berechnung
valk, gradk, hessk = f(x0)
log['x_list'].append(x0)
log['val_list'].append(valk)
log['norm_grad_list'].append(np.linalg.norm(gradk))

while True:
    # Newton Schritt
    #alt
    pk = np.linalg.solve(hessk, -gradk)
    alphak = armijo(f, pk, xk, rho, c1)
    xkplus1 = (xk + alphak * pk).ravel()

    #neu
    valkplus1, gradkplus1, hesskplus1 = f(xkplus1)

    # Abbruchbedingung
    if gill_murray_wright(xk, xkplus1, valk, valkplus1, gradk, k, tau, eps, kmax):
        break
    # Update
    #angepasst/umsortiert & hinzugefügt
    k += 1
    xk = xkplus1
    valk = valkplus1
    gradk = gradkplus1
    hessk = hesskplus1
    # Logging
    log['x_list'].append(xk)
    log['val_list'].append(valk)
    log['norm_grad_list'].append(np.linalg.norm(gradk))

log['xnv'] = xk
log['knv'] = k
# END SOLUTION
return log

```

```

def rosenbrock(x, y):
    """
    Implementierung der Rosenbrock-Funktion  $f(x, y) = 100(y-x^2)^2 + (1-x)^2$ 

    Input:
        x, y: Skalare bzw. Arrays zur Auswertung der Funktion.

    Output:
        val: Zielfunktionswert(e),
        grad: Gradient(en),
        hess: Hessematri(x/zen).
    """
    val, grad, hess = None, None, None

```

```

# TODO. Aufgabenteil 2.ii. Rosenbrock implementieren.
# BEGIN SOLUTION
# Berechne Wert
x = np.asarray(x)
y = np.asarray(y)
val = 100 * (y - x**2)**2 + (1 - x)**2

# Berechne Gradienten
# Differenzieren nach x:  $2 * 100 * (y - x^2) * (-2) * x - 2 * (1 - x)$ 
gradx = -400 * x * (y - x**2) - 2 * (1 - x)
# Differenzieren nach y:  $2 * 100 * (y - x^2) * 1$ 
grady = 200 * (y - x**2)
grad = np.array([gradx, grady])

# Berechne Hesse-Matrix
hesse_xx = 1200 * x**2 - 400 * y + 2
hesse_yy = 200 * np.ones_like(x)
hesse_xy = -400 * x

hess = np.stack([[hesse_xx, hesse_xy], [hesse_xy, hesse_yy]], axis=0)
# END SOLUTION

return val, grad, hess

if __name__ == '__main__':
    # TODO. Aufgabenteil 2.iii. Rosenbrock visualisieren.
    # BEGIN SOLUTION
    x = np.linspace(-3, 3, 1000)
    y = np.linspace(-3, 3, 1000)
    X, Y = np.meshgrid(x, y)
    Z, _, _ = rosenbrock(X, Y)

    fig = plt.figure()
    ax = fig.add_subplot(projection='3d')
    surf = ax.plot_surface(X, Y, Z, cmap=cm.hsv, linewidth=0, antialiased=False)
    fig.colorbar(surf, shrink=.5, aspect=10, pad=.1)
    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('$f(x, y)$')
    ax.set_title('Rosenbrock Funktion')

    plt.show()
    # END SOLUTION

    xstar = np.array([1, 1])
    rho = .5
    c1 = 1e-3
    eps = 1e-10
    kmax = 1000

    f = lambda x: rosenbrock(*x)

    for x0 in [np.array([0, -0.71]), np.array([-1.2, 1])]:
        for which_method in ['Gradientenabstieg', 'Newton']:

```

```

for tau in [1e-2, 1e-10]:
    print('----')
    # TODO: Aufgabenteil 3: which_method Verfahren testen und Ergebnisse in Konsole ausgeben
    # BEGIN SOLUTION
    start_time = time.time()
    if which_method == 'Gradientenabstieg':
        log = gradient_descent_erweitert(f,x0,rho,c1,eps,tau,kmax,xstar)
        anzahl_iter = log['kgd']
        loesung = log['xgd']
    else:
        log = newton_erweitert(f,x0,rho,c1,eps,tau,kmax,xstar)
        anzahl_iter = log['knv']
        loesung = log['xnv']

    gesamt_time = time.time() - start_time
    zeit_iter = gesamt_time/anzahl_iter

    print(f"Verfahren: {which_method}")
    print(f"x0: {x0}")
    print(f"Tau: {tau}")
    print(f"Berechnete Lösung: {loesung}")
    print(f"Anzahl Iterationen: {anzahl_iter}")
    print(f"Gesamte Zeit: {gesamt_time}")
    print(f"Durchschnittliche Zeit pro Iteration: {zeit_iter}")
    print()
    # END SOLUTION

    # TODO: Aufgabenteil 4: Iterationsverlauf plotten
    # BEGIN SOLUTION
    title = f"Plot für das {which_method}-Verfahren mit Startwert {x0} und Tau = {tau}"
    figure = plot_iteration_rosenbrock(log, title, rosenbrock)
    plt.show()
    # END SOLUTION

# TODO: Aufgabenteil 4: Diskussion
# BEGIN SOLUTION
print("Das erweiterte Gradientenabstiegsverfahren benötigt deutlich mehr Schritte als das Newton-Verfahren. Für das kleinere Tau wird auch erst durch die kmax Abbruchbedingung beendet.\n"
      "Das Newton-Verfahren benötigt deutlich weniger Iterationen und auch der Unterschied zwischen den Iterationswerten ist erwartbar (aufgrund der lokalen quadratischen Konvergenz) ist in der Anzahl der Iterationen geringer.\n"
      "Das Gradientenverfahren ist in den ersten Iterationen deutlich schneller als im Laufe des Verfahrens. Logischer Weise ist dadurch auch die Laufzeit von Newton besser als die vom Gradientenabstieg, wobei die Laufzeit je Iteration sich nicht so stark unterscheidet.\n"
      "Auch fällt auf, dass die Meldung, dass die Armijo-Bedingung nicht erfüllt ist, nie ausgegeben wird.")
# END SOLUTION

```

#TERMINAL OUTPUT:

Die Verbesserung der Werte, der x_k und der Norm des Gradienten sind sehr klein.

Verfahren: Gradientenabstieg

x0: [0. -0.71]

Tau: 0.01

Berechnete Lösung: [0.82877564 0.6864495]

Anzahl Iterationen: 423

Gesamte Zeit: 0.1474437713623047

Durchschnittliche Zeit pro Iteration: 0.0003485668353718787

Die maximale Anzahl an Iterationen ist erreicht.

Verfahren: Gradientenabstieg

x0: [0. -0.71]

Tau: 1e-10

Berechnete Lösung: [0.92010223 0.84599152]

Anzahl Iterationen: 1000

Gesamte Zeit: 0.3535034656524658

Durchschnittliche Zeit pro Iteration: 0.0003535034656524658

Die Norm des Gradienten ist kleiner epsilon.

Verfahren: Newton

x0: [0. -0.71]

Tau: 0.01

Berechnete Lösung: [0.99994377 0.9998843]

Anzahl Iterationen: 13

Gesamte Zeit: 0.0030205249786376953

Durchschnittliche Zeit pro Iteration: 0.00023234807527982272

Die Verbesserung der Werte, der xk und der Norm des Gradienten sind sehr klein.

Verfahren: Newton

x0: [0. -0.71]

Tau: 1e-10

Berechnete Lösung: [0.99999996 0.99999992]

Anzahl Iterationen: 14

Gesamte Zeit: 0.0029761791229248047

Durchschnittliche Zeit pro Iteration: 0.00021258422306605747

Die Verbesserung der Werte, der x_k und der Norm des Gradienten sind sehr klein.

Verfahren: Gradientenabstieg

x_0 : [-1.2 1.]

Tau: 0.01

Berechnete Lösung: [0.93062141 0.86614777]

Anzahl Iterationen: 51

Gesamte Zeit: 0.01994633674621582

Durchschnittliche Zeit pro Iteration: 0.00039110464208266316

Die maximale Anzahl an Iterationen ist erreicht.

Verfahren: Gradientenabstieg

x_0 : [-1.2 1.]

Tau: 1e-10

Berechnete Lösung: [0.97277594 0.9460233]

Anzahl Iterationen: 1000

Gesamte Zeit: 0.37281060218811035

Durchschnittliche Zeit pro Iteration: 0.00037281060218811036

Die Norm des Gradienten ist kleiner epsilon.

Verfahren: Newton

x_0 : [-1.2 1.]

Tau: 0.01

Berechnete Lösung: [0.99947938 0.99894834]

Anzahl Iterationen: 19

Gesamte Zeit: 0.0029914379119873047

Durchschnittliche Zeit pro Iteration: 0.00015744410063091077

Die Verbesserung der Werte, der x_k und der Norm des Gradienten sind sehr klein.

Verfahren: Newton

x_0 : [-1.2 1.]

Tau: 1e-10

Berechnete Lösung: [0.99999889 0.99999751]

Anzahl Iterationen: 20

Gesamte Zeit: 0.002962350845336914

Durchschnittliche Zeit pro Iteration: 0.0001481175422668457

Das erweiterte Gradientenabstiegsverfahren benötigt deutlich mehr Schritte als das Newton-Verfahren und wird für das kleiner Tau auch erst durch die kmax Abbruchbedingung beendet.

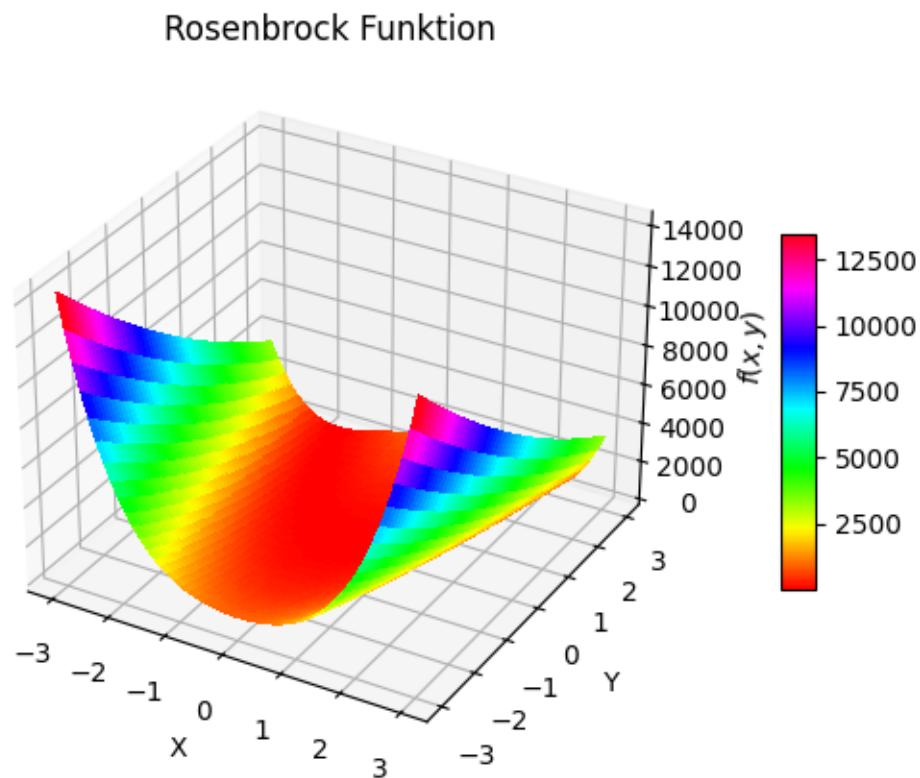
Das Newton-Verfahren benötigt deutlich weniger Iterationen und auch der Unterschied zwischen den beiden Tau ist erwartbar (aufgrund der lokalen quadratischen Konvergenz) ist in der Anzahl der iterationen nur sehr gering.

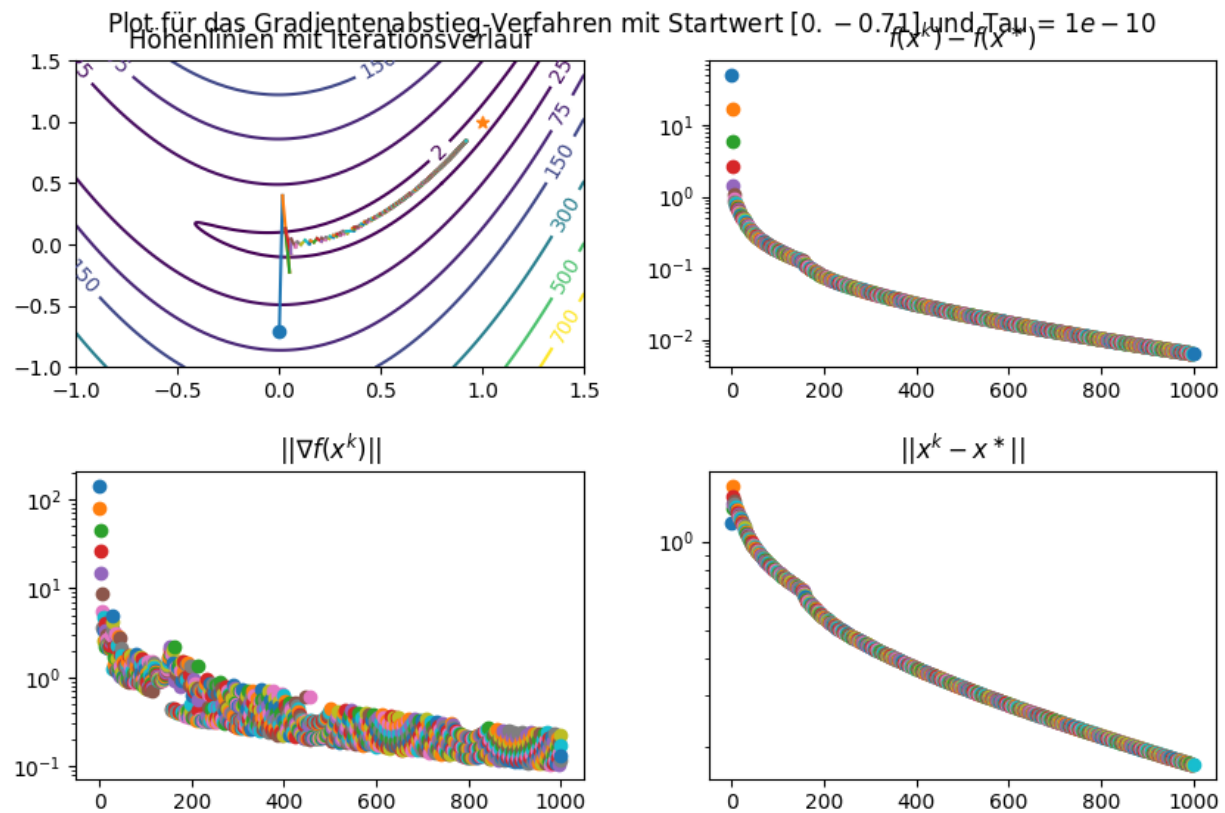
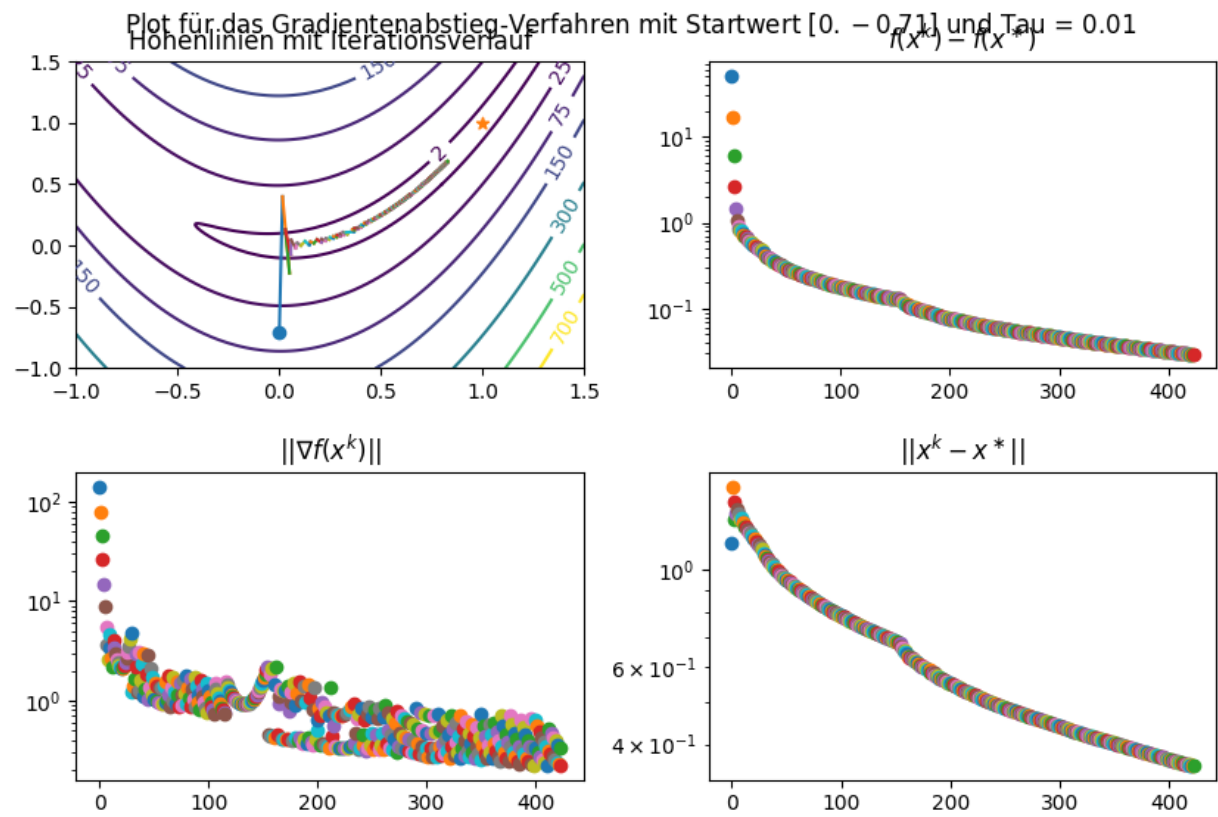
Das Gradientenverfahren ist in den ersten Iterationen deutlich schneller als im Laufe des Verfahrens.

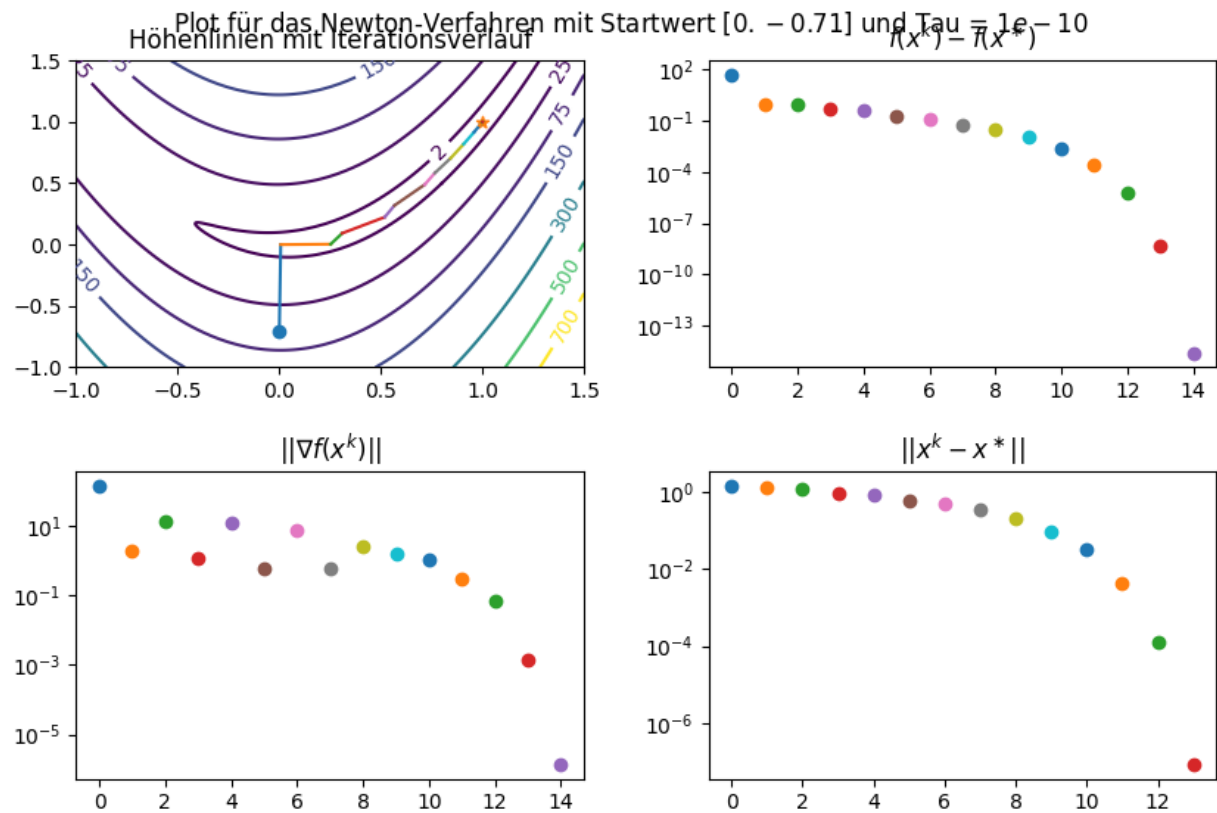
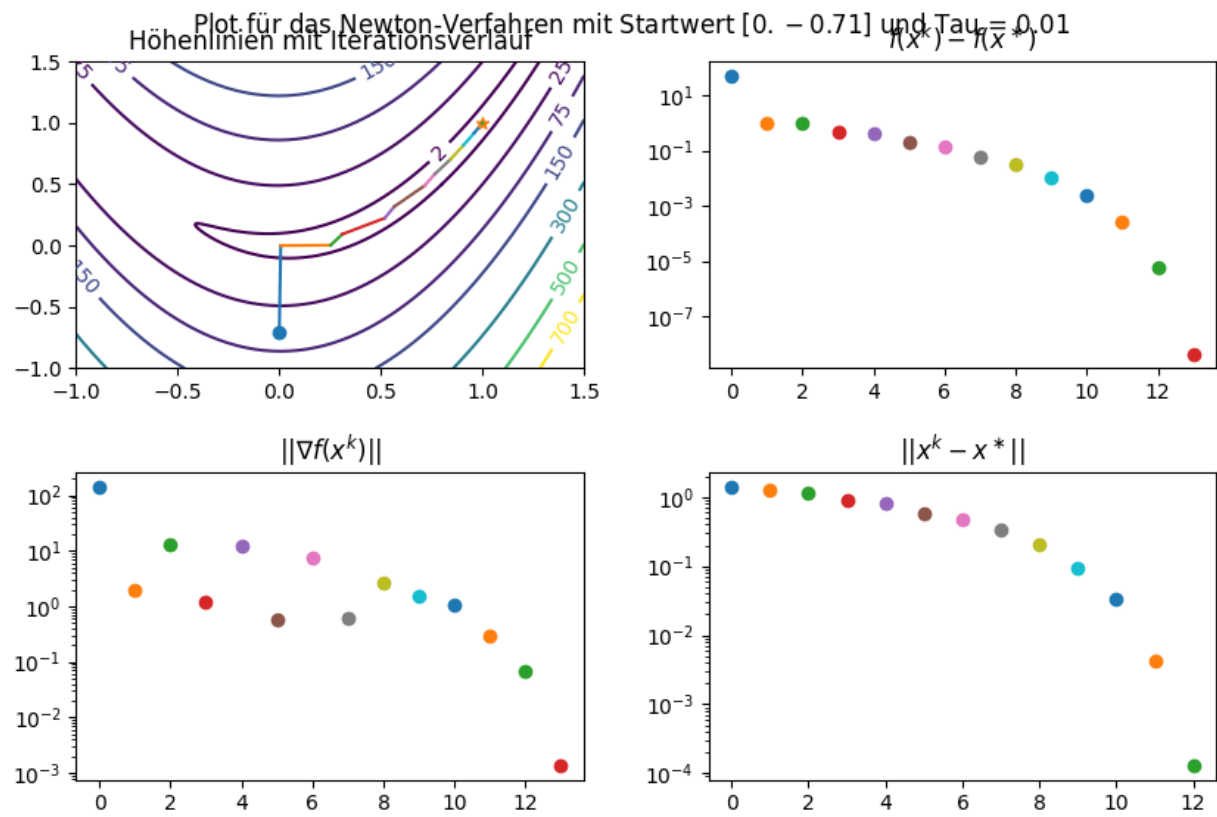
Logischer Weise ist dadurch auch die Laufzeit von Newton besser als die vom Gradientenabstieg, wobei die Laufzeit je Iteration sich nicht so stark unterscheidet.

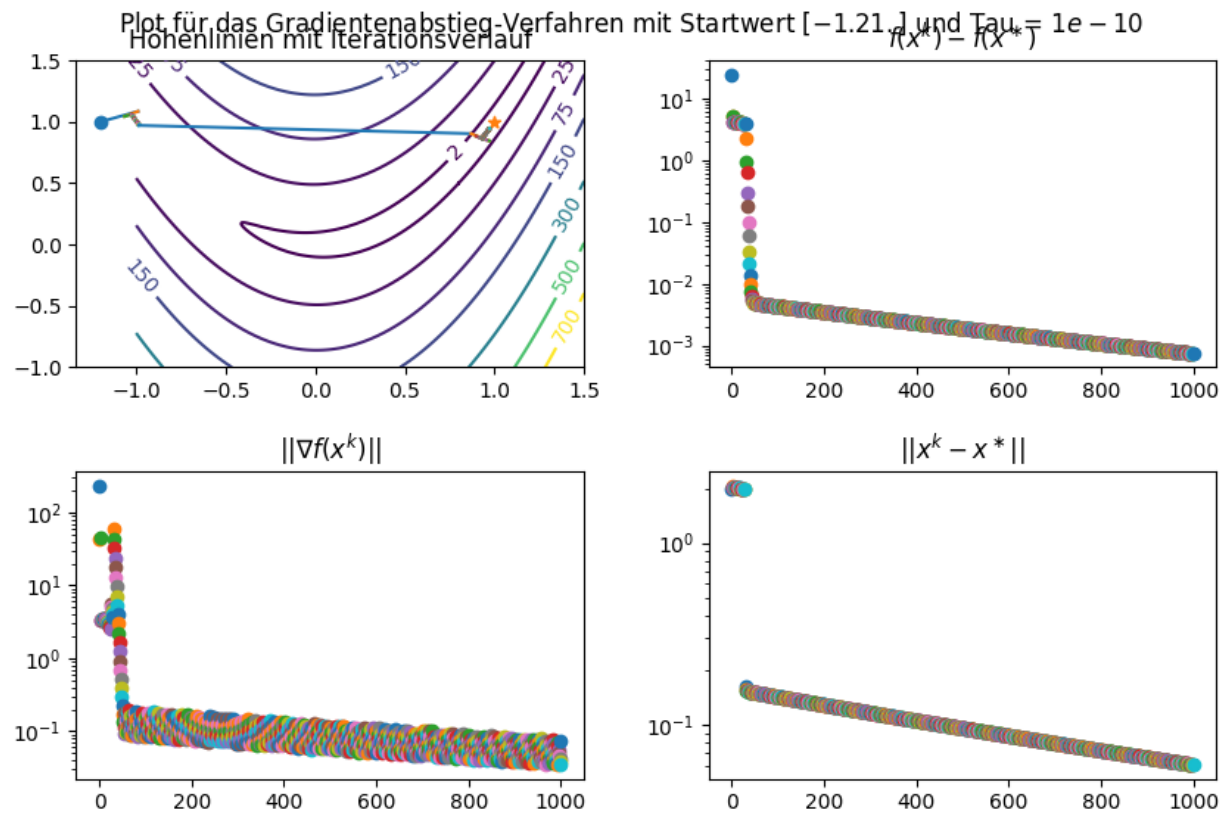
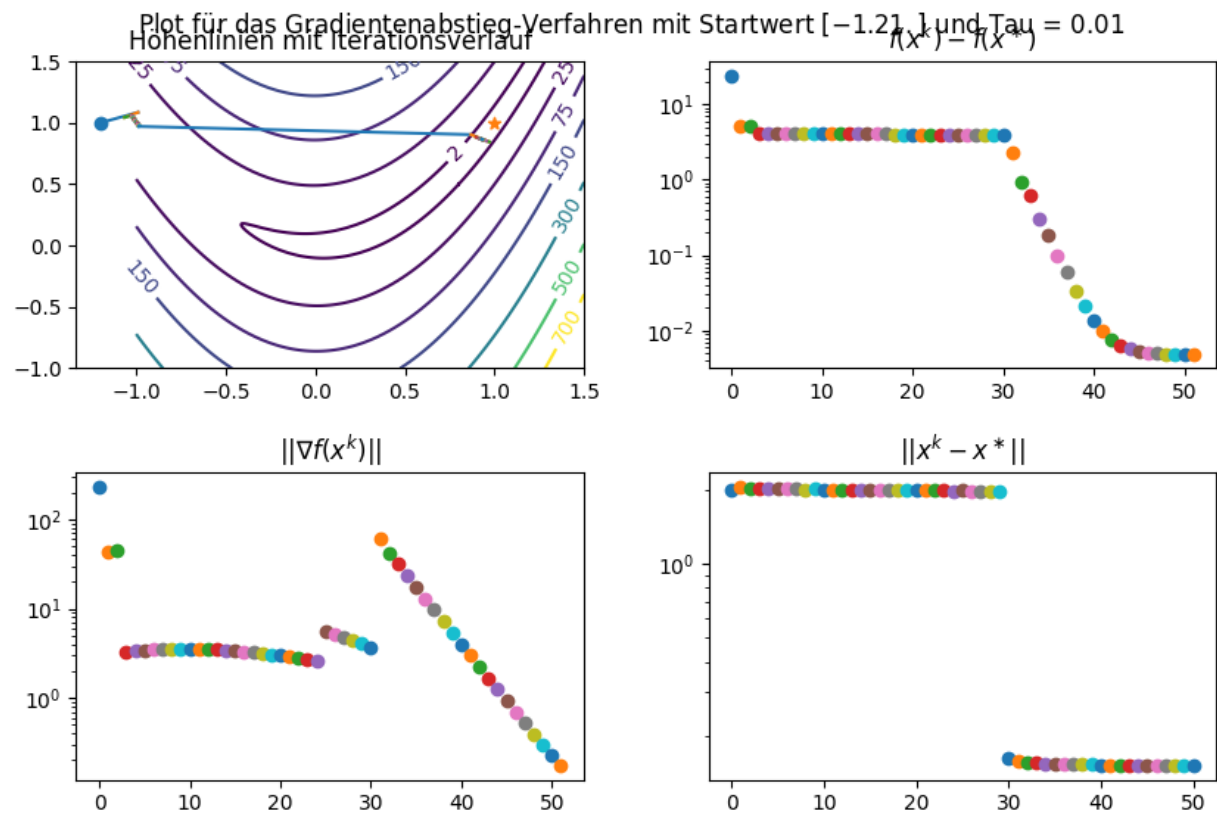
Auch fällt auf, dass die Meldung, dass die Armijo-Bedingung nicht erfüllt ist, nie ausgegeben wird.

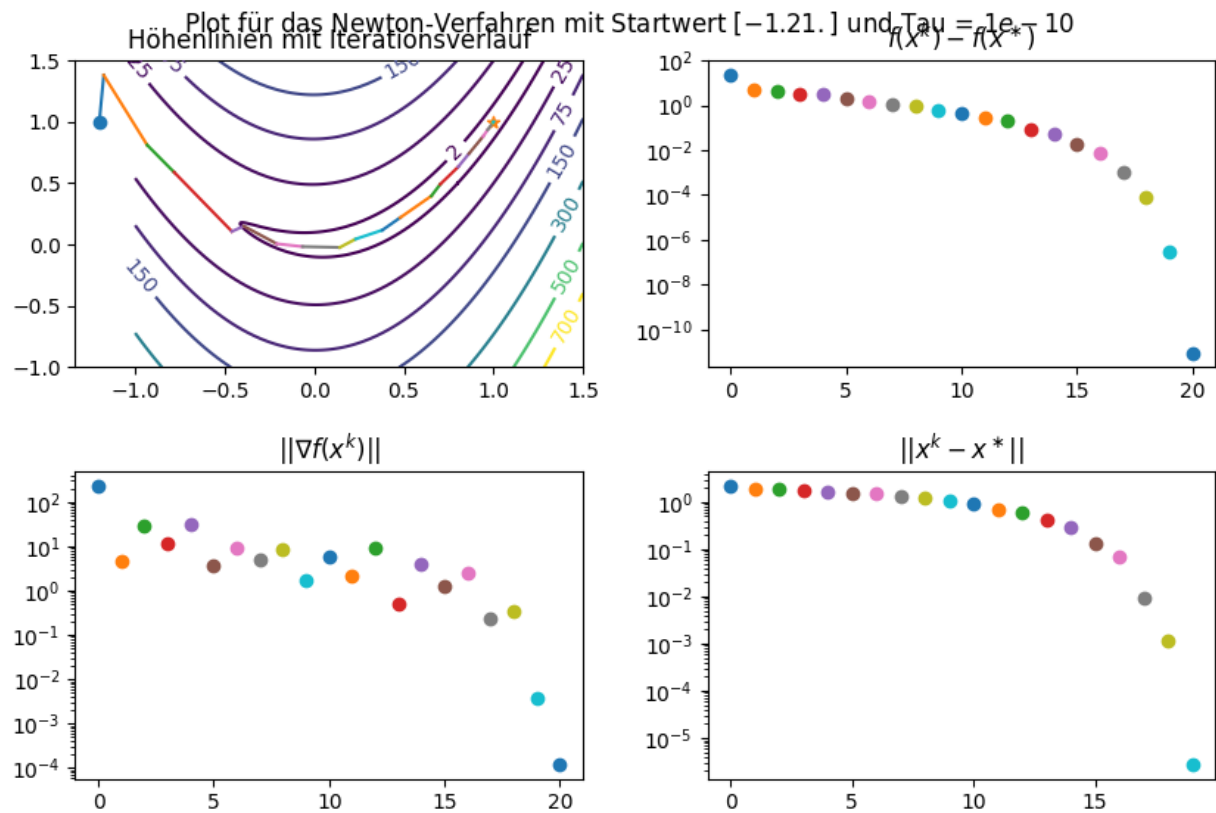
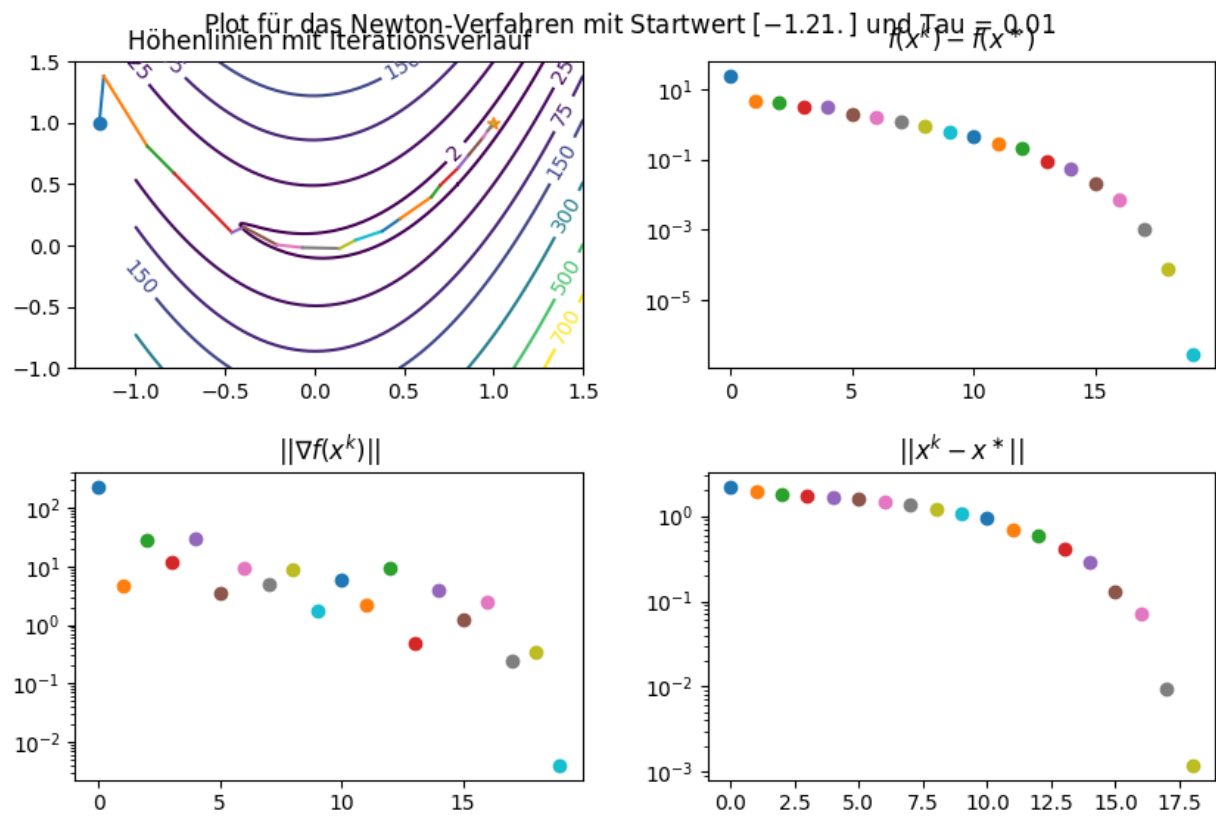
#PLOTS:











```

import numpy as np
import matplotlib.pyplot as plt

def plot_iteration_rosenbrock(log, title, rosenbrock):
    """
    Funktion zur Erstellung von Plots zum Optimierungsverlauf der Rosenbrock-Funktion anhand der Ergebnisse.
    Im ersten Plot wird der Iterationsverlauf auf einem Surface-Plot dargestellt, in weiteren 3 Konvergenzplots
    Die Funktion liefert fig zurück. Stellen Sie diese dar, indem Sie plt.show() in Ihrer Hauptdatei aufrufen.

    Input:
        log: Dictionary, das mindestens folgende Einträge enthält:
            log = {
                'x0': Vektor,                # Startwert
                'x_list': Liste,              # Iterierte
                'val_list': Liste,            # Funktion ausgewertet an den Iterierten
                'norm_grad_list': Liste,      # Norm des Gradienten ausgewertet an den Iterierten
            }
        title: Titel des Plots.

    Output:
        fig: hergestellte Figure mit den Plots
    """

    # Figure initialisieren
    fig, ax = plt.subplots(2, 2)
    fig.set_size_inches(9, 6)
    fig.tight_layout(pad=3, h_pad=3)
    fig.suptitle(title)

    # Surface-Plot mit Iterationsverlauf
    x = np.linspace(-1., 1.5, 651)
    xx, yy = np.meshgrid(x, x)
    zz = rosenbrock(xx.flatten(), yy.flatten())[0].reshape((651, 651))
    CS = ax[0, 0].contour(xx, yy, zz, levels=[2, 25, 75, 150, 300, 500, 700])
    ax[0, 0].clabel(CS, inline=1, fontsize=10)
    ax[0, 0].scatter(*log['x0'], marker='o')
    ax[0, 0].scatter(1, 1, marker='*')
    for k in range(len(log['x_list']) - 1):
        xk = log['x_list'][k]
        xkplus1 = log['x_list'][k + 1]
        ax[0, 0].plot([xk[0], xkplus1[0]], [xk[1], xkplus1[1]])
    ax[0, 0].set_title('Höhenlinien mit Iterationsverlauf')

    # Plotten der Zielfunktionsfehler
    val_list = log['val_list']
    val_star = 0
    for k, valk in enumerate(val_list):
        ax[0, 1].scatter(k, valk - val_star)
    ax[0, 1].set_yscale('log')
    ax[0, 1].set_title(r'$f(x^k) - f(x^{\ast})$')

```

```

# Plotten der Gradientennorm
norm_grad_list = log['norm_grad_list']
for k, norm_gradk in enumerate(norm_grad_list):
    ax[1, 0].scatter(k, norm_gradk)
ax[1, 0].set_yscale('log')
ax[1, 0].set_title(r'$||\nabla f(x^k) ||$')

# Plotten der Konvergenz der Iterierten
x_list = log['x_list']
xstar = np.array([1, 1])
for k, xk in enumerate(x_list[1:]):
    ax[1, 1].scatter(k, np.linalg.norm(xk - xstar))
ax[1, 1].set_yscale('log')
ax[1, 1].set_title(r'$||x^k - x^{\ast}||$')

return fig

```

#TERMINAL OUTPUT:

#PLOTS: